



SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF COMPUTER, CONTROL AND MANAGEMENT
ENGINEERING

G1 at MEF
AN HRAI FRAMEWORK FOR PUBLIC ADMINISTRATION

Professors:

Luca Iocchi

Vincenzo Suriani

Student:

Alessandro Massari

Matricula: 22050305

Academic Year 2025/2026

Contents

1	Introduction	3
1.0.1	Problem and contextual relevance	3
1.0.2	Our system and general architecture	3
2	Related works	5
3	Design	7
3.1	Users, interactions and functional requirements	7
3.2	Hardware infrastructure	7
3.3	Computing and AI deployment	9
3.4	Software and functional design	9
4	Solution	11
4.1	Scene Generation	12
4.2	Perception Layer: LOST-3DSG	13
4.3	Semantic Layer	14
4.4	Locomotion	16
4.5	Human-G1 interaction	18
5	Implementation	21
5.1	Scene Generation	22
5.2	LOST-3DSG integration	23
5.2.1	Integration challenges and solutions applied	23
5.2.2	Architectural refactoring of perception.py	24
5.3	Object manager and semantic layer implementation	25
5.3.1	Semantic map	28
5.4	Locomotion implementation	29
5.4.1	Occupancy map	29
5.4.2	Semantic goal selection and path planning	30
5.4.3	Waypoint navigation	32
5.4.4	Walking in MjLab	33
5.5	G1-Humans interaction	33
5.5.1	Natural Language task interpretation	34
5.5.2	Semantic grounding and task answer	35
5.5.3	Runtime integration	36
6	Experimental Evaluation	38
6.1	Navigation task evaluation	38
6.2	Location task evaluation	39
6.3	Description task evaluation	40

7	Results	42
7.1	Exploration results	42
7.2	Interaction and task execution results	44
7.2.1	Reasoning over multiple object instances	44
7.2.2	Answering under incomplete semantic knowledge	45
7.2.3	Navigation behaviour	46
7.3	Video demonstrations	46
8	Conclusions	48
8.1	Challenges	48
8.2	Strengths and limitations	49
8.3	Future improvements	50
	References	51

1 Introduction

MEF-G1 is an embodied Human-AI-Robot Interaction (HRAI) framework developed up to the Unitree G1 humanoid, conceptualized as a situated conversational assistant for Public Administration (PA) environments, specifically modeled on the Italian Ministry of Economy and Finance (MEF). The primary objective of this project is to examine how physical embodiment, multimodal spatial perception and dialogue management can enhance public service workflows, bridging the gap between bureaucratic procedures and accessible civic interfaces.

1.0.1 Problem and contextual relevance

Public administration facilities represent a demanding and socially nuanced testbed for Human-Robot and Artificial Intelligence (HRAI) research. Unlike controlled laboratory setups or commercial retail floors, institutional offices bring together a diverse user base characterized by broad demographic spectrums, disparate digital literacy levels, and contrasting operational expectations. In this high-stakes context, interaction design transcends mere technical execution, directly influencing institutional trust, perceived authority, and information accessibility.

Currently, disembodied conversational agents, such as Microsoft Copilot and Google Gemini, represent the default paradigm deployed by major IT consulting firms and enterprise AI vendors to automate daily administrative workflows. However, these interfaces present substantial operational and relational limitations. On an interpersonal level, they lack emotional, nonverbal signaling, and genuine empathetic resonance, which are essential when mediating bureaucratic anxiety or guiding non-expert citizens through rigid institutional protocols. More fundamentally, these models operate in total detachment from the physical world. Without spatial grounding and no embodied knowledge base of the environment, they remain completely unaware of room geometry, human proximity, directional gestures, and the physical location of furniture. Consequently, investigating whether a physically humanoid platform can overcome these perceptual and socio-emotional blind spots is essential for deploying autonomous AI responsibly within civic infrastructure and collaborative workspaces.

1.0.2 Our system and general architecture

To explore these dynamics, we developed an integrated software architecture simulating the deployment of the Unitree G1 within a high-fidelity replica of an administrative workspace. Built upon the ROS 2 middleware, the system couples real-time sensory acquisition with Natural Language Processing and spatial semantic mapping with the aim to answer two primary use cases:

- **Office Worker Assistance:** enabling administrative personnel to query procedural documentation, verify internal regulations, and locate office resources using natural language.
- **Visitor Reception and On-boarding:** welcoming citizens and visiting staff, identifying their administrative needs, providing directional guidance, and assisting new employees through orientation protocols.

Conducting empirical evaluations directly within an active PA facility poses substantial logistical, safety, and operational challenges as we already know. Consequently, in this stage the system has been deployed into a virtual simulation environment is essential for rigorous, risk-free validation prior to physical execution.

In details, to balance physical accuracy with experimental agility, the framework is built upon MuJoCo, specifically through Mjlab. While MuJoCo delivers industry-standard contact dynamics, continuous collision handling, and numerical stability for humanoids with an high number of DOF like the Unitree G1, mjlab provides the critical integration layer needed for embodied AI research. It bridges raw physics calculation with modular robotics workflows, offering streamlined asset parsing, GPU-accelerated sensor rendering, and standardized interfaces for humanoid control and observation spaces. This setup allows realistic visual and depth streams to be synthesized alongside low-level joint states without the overhead of ad-hoc physics wrappers. Building upon this simulated foundation, the selected technical stack is tailored to resolve the core challenges of situated, grounded interaction:

- **Physics Simulation Decoupled via ROS2:** Decoupling the simulation back-end from cognitive processing via the ROS 2 middleware ensures deterministic, reproducible assessment of locomotion, head tracking, and perception pipelines. This modular separation mirrors real hardware topologies, safeguarding the system against platform instability while preserving a direct, bidirectional sim-to-real transfer pipeline.
- **VLMs and Open-Vocabulary Detector:** to avoid relying on rigid closed-set classifiers limited to predefined object categories, integrating VLMs and Open Vocabulary detection models allows the robot to parse unstructured environments dynamically. This enables zero-shot recognition of diverse administrative artifacts, office supplies, and architectural features directly through natural language descriptors.
- **3D Semantic Scene Graphs:** By generating a dynamic 3D semantic scene graph that links visual-linguistic tokens to metric Cartesian bounding boxes and topological edges, the system gains geometric and *situational awareness*. This spatial grounding enables the robot to effectively reason about physical dimensions, support surfaces, and proximity relative to human users and office assets.

2 Related works

The development of the project was influenced both by the material presented during the Human-Robot Interaction course and by recent research on semantic scene representations, embodied AI and world models.

A first important reference was the course material on robot embodiment presented by Prof. L. Iocchi. In particular, the lectures introduced a user-centred, top-down design methodology in which the design process starts from the target users and expected interactions, followed by morphology and appearance, required robot functionalities, and finally the hardware platform. This methodology contrasts with a hardware-first, bottom-up design approach and directly influenced the design choices of the proposed system. The MEF office assistant scenario and the required interaction capabilities were therefore defined before selecting the specific humanoid platform and implementation technologies. The same lectures also highlighted the importance of matching robot form, capabilities, and user expectations, which motivated the decision to expose only behaviours that are actually supported by the system.

Another central influence came from the lectures by Prof. V. Suriani on semantic scene understanding and 3D scene graphs. These lectures introduced the idea of representing the environment through persistent object-centric entities and relations rather than relying only on raw geometric maps. In particular, LOST-3DSG was presented during the course and later became the main reference for the semantic mapping component adopted in this project. LOST-3DSG proposes a lightweight open-vocabulary 3D scene graph for semantic object tracking in dynamic environments.

A second research direction considered during the early design phase was the use of learned world models. In this context, V-JEPA-2 by Meta [1] represented an important reference. V-JEPA-2 investigates how self-supervised learning from large-scale video data can produce representations capable of supporting understanding, prediction, and planning in the physical world. The work additionally introduces V-JEPA-2-AC, an action-conditioned world model obtained by post-training the learned representation with a comparatively small amount of robot interaction data.

The original idea for this project included investigating whether such a learned world model could complement or partially replace explicit semantic representations for robot perception and planning. This integration was not completed within the available development time, and the final system therefore adopts an explicit persistent semantic world model. Nevertheless, V-JEPA-2 remains particularly relevant as a possible future extension, especially for temporal reasoning, prediction of scene dynamics, and model-based planning. Additionally, the work on π_0 , a Vision-Language-Action model for general robot control [2] provided a complementary perspective on the integration of foundation models and embodied robot behaviour. π_0 combines a pre-trained vision-language backbone with a robot-specific action expert and uses flow matching

to generate continuous action sequences. The model is trained across multiple robot embodiments and tasks, with the goal of transferring semantic knowledge from large-scale vision-language pre-training to physical robot control. Although our architecture does not use an end-to-end VLA policy, the π_0 work was useful in framing the relationship between high-level semantic reasoning and low-level physical execution. In particular, it reinforced the idea that language-conditioned reasoning can be separated from, but tightly integrated with, robot-specific control. In the proposed system this separation is explicit: the LLM handles task interpretation, the semantic representation grounds the requested entity, and the locomotion policy executes the resulting physical motion.

Overall, these works influenced different levels of the proposed architecture. The course material on embodiment guided the user-centred design methodology; LOST-3DSG motivated the persistent semantic world representation; V-JEPA 2 inspired the investigation of predictive world models; π_0 provided a reference for the integration of language, perception, and robot control and the European Robotics League methodology provided the basis for the experimental evaluation. In addition, the knowledge acquired during the Planning and Reasoning course by Prof. A. Marrella was useful for structuring the planning aspects of the system, while the material presented by Prof. G. Oriolo contributed to the locomotion and navigation components, in particular gait generation concepts and the construction of the navigation map, including obstacle inflation for safe path planning. In conclusion, the final system does not reproduce any of these approaches directly, but combines several of their underlying principles within a modular HRI architecture tailored to the MEF office scenario.

3 Design

The ideal robotic platform for the proposed scenario is a humanoid service robot able to move autonomously inside an office, perceive and maintain a semantic representation of the environment, understand natural-language requests, and interact with both employees and external visitors. The design should therefore combine reliable humanoid locomotion, multimodal perception, sufficient computational resources for AI-based reasoning, and a modular software architecture in which individual components can be replaced or extended without redesigning the complete system.

3.1 Users, interactions and functional requirements

Following a user-centred design approach, as seen during the HRAI course, the robot was first conceived starting from the expected users and interactions rather than from a specific hardware platform. The target scenario is a Public Administration office, where the robot may interact both with employees familiar with the environment, both with external visitors who may require guidance and more detailed information. Consequently, development have to address not only technical efficiency such as navigation reliability and perception accuracy, but also **social effectiveness**. The interaction behaviour should remain appropriate to the deployment context and to the user involved.

Core interaction requirements therefore include understanding natural-language requests, locating and describing relevant objects or workstations, guiding users through the environment, and adapting the level of explanation to the interaction context. Deriving from these requirements, key functional capabilities include multimodal perception, semantic mapping, natural-language understanding, autonomous navigation, grounded response generation, and humanoid locomotion.

Then, physical embodiment should support these functions without creating unrealistic expectations. A humanoid morphology is appropriate for the considered office scenario because it can operate in spaces designed for humans and provides an intuitive physical presence for face-to-face interaction, while the interaction capabilities exposed by the system remain explicitly constrained to the behaviours actually supported by the robot.

3.2 Hardware infrastructure

The **Unitree G1** was selected as the reference humanoid platform because it represents one of the most capable and accessible humanoid robots currently available for research and development. Its human-like morphology makes it particularly suitable for an office-oriented Human-Robot Interaction scenario, where the robot is expected to share spaces originally designed for people and to interact naturally with human users.

From a sensing perspective, the G1 integrates the *intelRealSense D4435i* depth camera and a 3D LiDAR, together with a microphone array and speaker, providing a suitable hardware basis for visual perception, navigation, and basic verbal interaction. The G1 EDU configuration additionally supports secondary development and optional high performance computing modules, including NVIDIA Orin based platforms. These characteristics make the platform particularly appropriate for the proposed architecture. However, the proposed design is intentionally not tied to the Unitree platform. Another suitable candidate would be the **Booster T2**, a more recent humanoid development platform, which can boast on an *NVIDIA Jetson Thor T5000* computing platform. The robot also exposes Ethernet, WiFi 6, and external development interfaces, making it well suited for embodied-AI applications.

Replacing the G1 with another humanoid such as the Booster T2 would therefore not require a complete redesign of the proposed system. Most of the perception, semantic reasoning, planning and interaction components are independent from the specific robot embodiment. The main adaptations would concern the hardware-specific interfaces: sensor calibration and coordinate frames, camera and depth-data acquisition, the robot footprint used for navigation, and the interface between the waypoint controller and the low-level locomotion system. In particular, the G1-specific velocity-tracking policy would have to be replaced by the corresponding locomotion controller available for the new platform. Anyway, thanks to the usage of ROS 2 middleware the transition from one robot to another should not be extremely painful.

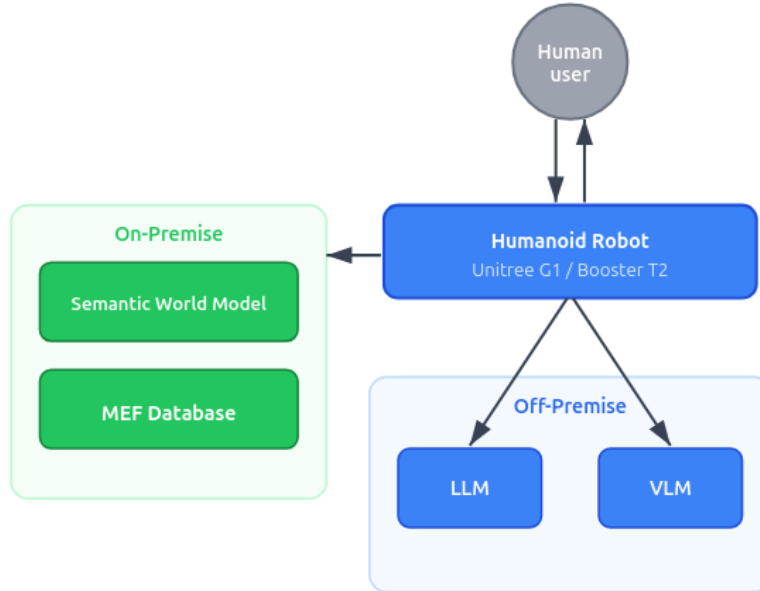


Figure 1: Possible deployment architecture. Humanoid robot provides sensing, real-time control, and physical interaction, while computationally intensive AI models can either run onboard, when sufficient computing resources are available, or as off-premise service. A connection to the Ministerial DB could be helpful in retrieving information about people and documents.

3.3 Computing and AI deployment

In the current prototype, cloud-hosted LLM and VLM are used for language interpretation, response generation, and parts of the semantic perception pipeline. This approach provides immediate access to powerful models but introduces an external dependency on Internet connectivity, API availability, latency, service quotas, and third-party providers. For a real deployment inside a Ministry, an ideal architecture would reduce this dependency by hosting the required AI models within the organisation’s own infrastructure following an on-premise architecture. A local inference server connected to the robot through the institutional network could execute the LLM and VLM components while keeping sensitive information within the local infrastructure and granting service continuity. A more advanced possibility is to execute part of the AI inference directly on the humanoid. This is technically feasible with suitable hardware configurations of both considered platforms. The G1 EDU supports optional high-compute modules such as NVIDIA Orin, while Unitree also offers G1 variants equipped with Orin NX-class computing hardware. This would make the execution of appropriately sized and optimised local language models technically possible, although larger models would still be more naturally hosted on a dedicated on-premise server. Considering Booster T2, the Professional version provides an even stronger option through the NVIDIA Jetson Thor T5000, explicitly intended for embodied-AI workloads and onboard perception and reasoning. Consequently, a future implementation could distribute computation between the robot and a local server according to the computational requirements of each model.

3.4 Software and functional design

At software level, the ideal robot follows the modular organization adopted throughout the project. Perception, persistent semantic mapping, semantic reasoning, Human-Robot Interaction, navigation, and locomotion are implemented as separate functional layers connected through explicit interfaces as described in the next sections.

This schema allows functionality to be added, removed, or replaced with limited impact on the rest of the system. For instance, replacing Gemini with a locally deployed LLM does not require modifications to the navigation subsystem, while changing the locomotion policy does not affect the Semantic Query Engine or the interaction logic. Similarly, the LOST-3DSG perception backend could be replaced by another perception framework as long as it provides a compatible persistent object representation.

Same principle is applied within the Human-Robot Interaction layer where language behaviour of the robot is partly specified through external prompt files rather than being hard-coded into the application logic. Files such as `task_prompt.txt` and the response-generation prompt can therefore be modified independently to introduce new

supported intents, alter the communication style, or adapt the robot to a different deployment scenario.

For example, the current intents *navigate*, *locate*, and *describe* could be extended with additional capabilities such as providing institutional information, answering questions about services, requesting clarification, accompanying visitors, or interacting differently according to the user’s role. Adding such capabilities would mainly require extending the task interpretation rules and implementing the corresponding task handler, rather than redesigning the complete interaction pipeline.

This modularity is particularly important for an HRI system deployed in a real institution, since the expected tasks and interaction policies are likely to evolve over time. The ideal robot should therefore not be designed as a fixed collection of behaviours, but as an extensible embodied-AI platform in which perception, reasoning, interaction, and action capabilities can be progressively updated while preserving a stable overall architecture.

4 Solution

The proposed framework follows a modular architecture in which existing locomotion and simulation components are integrated with a custom perception and semantic mapping pipeline developed during this project. The main objective is to enable the G1 humanoid to autonomously perceive, understand and navigate an indoor environment while maintaining a clear separation between low-level control and high-level semantic reasoning. Rather than modifying the internal implementation of the locomotion framework, our solution extends it through external modules connected via ROS2 communication. This design choice simplifies maintenance, improves modularity and allows each subsystem to be developed and tested independently. The overall architecture can be divided into the following main layers:

- **Scene Generation Layer:** custom scene builder dynamically generates the MuJoCo environment from a YAML scene description. Instead of manually editing XML files, the environment is automatically reconstructed by combining predefined objects and semantic information. During future developments, the YAML representation will be continuously updated according to the robot’s observations, allowing incremental reconstruction of previously unseen environments.
- **Perception Layer:** A virtual RGB camera has been mounted on the robot’s head to periodically acquires images of the simulated environment. Then a dedicated ROS 2 node publishes these images, reproducing the same communication interface that will be used on the physical robot. **This abstraction allows the perception pipeline to remain unchanged when moving from simulation to real hardware.**
- **Semantic Layer:** A second ROS 2 node subscribes to the camera stream and executes the semantic perception pipeline. Object detection will be performed using *OWLv2*, producing semantic labels along with their corresponding image locations. These detections are then processed by the Object manager, which maintains a persistent semantic representation of the environment. The Object Manager is responsible for creating new semantic entities, updating existing objects, handling repeated observations, and maintaining the consistency of the global scene representation.
- **Simulation and Locomotion Layer:** This layer is based on the MJLab framework, which provides the MuJoCo physics simulation together with an Actor-Critic reinforcement learning policy trained for velocity tracking of the Unitree G1 humanoid robot.

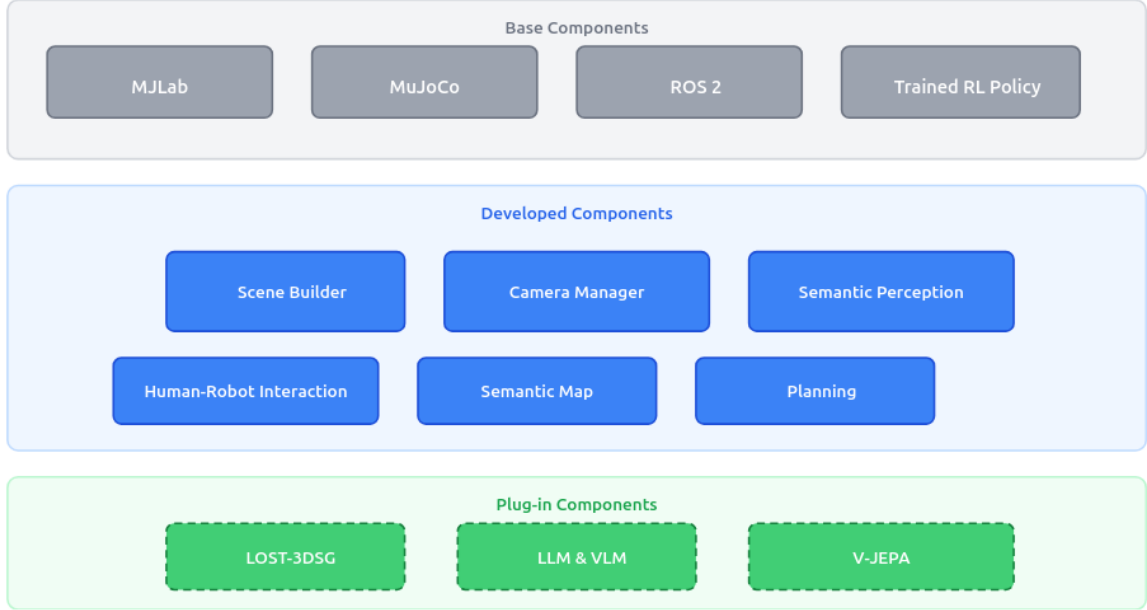


Figure 2: Overall system components

A key design principle of the proposed architecture is the complete decoupling between robot control and perception. The locomotion pipeline operates independently from the semantic reasoning pipeline, while communication between the two subsystems occurs exclusively through ROS 2 topics. This asynchronous architecture prevents computationally expensive perception algorithms from affecting the control frequency of the robot and closely resembles the software architecture typically adopted on real robotic platforms. The resulting system therefore consists of two largely independent execution pipelines: The *control pipeline*, responsible for simulation, locomotion and camera acquisition; the *perception pipeline*, responsible for object recognition, semantic map construction and future navigation planning. This modular organization, as stated above, considerably simplifies future extensions. In particular, replacing the simulated RGB camera with the physical camera mounted on the robot or substituting OWLv2 with different vision-language models will require modifications only within the corresponding module, without affecting the locomotion or simulation components. Likewise, future integration of V-JEPA for world modelling and semantic prediction can be achieved by extending the perception layer while preserving the remainder of the architecture.

4.1 Scene Generation

Given the practical hurdles of deploying and testing on a physical G1 robot, we opted to construct a virtual environment that closely mirrors reality with true to life dimensions and by incorporating IKEA furniture. Nevertheless, a *sim-to-real* transfer will inevitably be required to validate our approach in practice. In details,

the scene-generation pipeline was designed to provide a clear separation between the semantic description of the environment, the visual representation of objects, and their physical representation in MuJoCo. Objects are first described in a YAML scene configuration, where each element specifies attributes such as its name, position, type, dimensions, and, for complex objects, the path to the corresponding .glb mesh asset. The developed `build_scene.py` script parses this YAML description and automatically generates the MuJoCo XML scene, creating the world, floor, lighting, object bodies, and corresponding geometries. For objects represented by external 3D models, the architecture deliberately separates the Visual geometry, which contains the detailed IKEA mesh and is used exclusively for rendering, from the Collision geometry, represented by one or more simple primitives such as boxes and used exclusively for physics and collision detection. This separation makes it possible to preserve realistic visual appearance without introducing the computational and numerical complexity of detailed mesh-based collisions. The developed components therefore act as a bridge between the project-specific YAML scene description and the existing MJLab simulation environment, while relying on existing MuJoCo mechanisms for rendering, collision handling, bodies, joints, and assets.

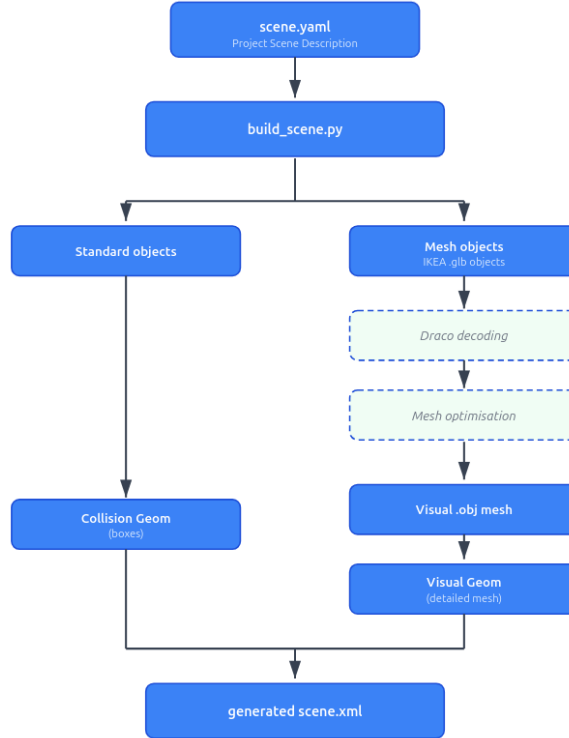


Figure 3: Scene generation pipeline: from .yaml to .xml format

4.2 Perception Layer: LOST-3DSG

The perception layer provides the robot with a persistent spatial-semantic representation of the simulated office environment. Its role is to detect objects in RGB and Depth

frames, associate observations over time and maintain a structured world representation that can be queried by the higher-level interaction and navigation modules. The proposed architecture is based on the integration of the **open vocabulary** framework LOST-3DSG [3] with the simulated sensing pipeline of the G1 robot. The perception stack combines RGB and depth data, camera calibration information, spatial transformations, object detection, segmentation, Vision-Language Model reasoning (VLM), and 3D geometric reconstruction in order to obtain persistent object instances enriched with semantic attributes.

Starting from the input side, the G1 head camera provides RGB-D observations from the MuJoCo simulation. These data are bridged to ROS 2 together with the camera pose and intrinsic parameters, allowing the perception subsystem to express detections in a common global coordinate frame. The data are then processed by the integrated LOST-3DSG pipeline, which performs open-vocabulary object detection, segmentation, local semantic description, and 3D geometric estimation. Finally, the resulting observations are not treated as independent frame-level detections. Instead, they are associated with previously observed objects and incorporated into a persistent representation of the environment. This persistent world model stores semantic labels, descriptive attributes, and spatial information for each object instance, allowing the system to preserve environmental knowledge across multiple perception cycles.

Several components of the perception architecture were specifically developed or adapted within the project. These include the MuJoCo to ROS2 sensing bridge, the spatial transformation interface between the simulator and the ROS2 TF system, the perception orchestration logic, the asynchronous processing architecture, the Gemini-based VLM backend and the persistent semantic integration layer. Conversely, LOST-3DSG, the underlying object-detection and segmentation models, ROS 2 communication infrastructure, and the MuJoCo camera simulation were integrated as existing components.

4.3 Semantic Layer

Within the proposed architecture, the **Object Manager** acts as an integration and orchestration layer between the output of the perception system and the semantic representation used by the remaining components of the architecture. In particular, it will be responsible for receiving the processed scene information, determining whether an observation corresponds to an already known entity or to a newly detected object, and coordinating the update of the semantic map. The semantic mapping module will maintain the persistent state of the environment, including object information and, when available, spatial and semantic relationships between entities.

As described above, the perception subsystem provides persistent object instances enriched with geometric and semantic information, including object labels, 3D bounding boxes, spatial positions, and descriptive attributes. The semantic layer builds on

this representation to reason about relationships between objects and to identify meaningful entities in the environment. Rather than relying only on absolute coordinates, the system uses spatial relations such as proximity and support to describe how objects are organized within the office as described in the next section.

This relational representation is particularly useful in environments containing several visually similar objects. For example, multiple desks may be present in the same office, but one of them can be identified as the *director desk* because it is located near a specific chair or bookshelf and supports characteristic objects such as a mug. In this way, the semantic layer associates high-level task concepts with the physical objects currently represented in the persistent map. Then, to keep symbolic knowledge separate from the continuously updated geometric representation, task-oriented semantic information is stored in a lightweight knowledge base. The knowledge base defines semantic entities and their task-relevant relationships, while the persistent perception model stores the physical state of the environment. A semantic query engine connects the two representations by evaluating the symbolic constraints against the perceived objects and retrieving the physical instance that best matches the requested target. Looking forward, this separation between symbolic knowledge and perceived geometry also provides a natural direction for future development. Instead of relying only on a manually defined knowledge base, the robot could progressively 'update its memory' by extending its semantic knowledge through situated interaction with office personnel. For example, it could ask users to clarify workstation ownership, desk assignments, or room functions, and then associate these new semantic concepts with the corresponding perceived entities. This would allow the semantic representation to adapt over time to changes in the working environment.

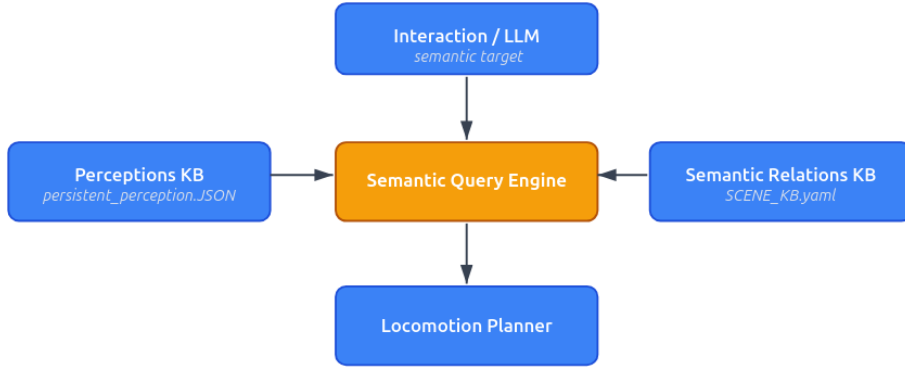


Figure 4: Semantic layer architecture

As shown in Fig.4, overall semantic reasoning flow starts from the target generated by the interaction subsystem. Then, its corresponding semantic concept is retrieved from the knowledge base and converted into a set of object properties and spatial relations. These relations are then evaluated against the persistent world representation produced by the perception pipeline. Once a matching physical object is found, its current

geometric information can be provided to downstream modules, for example as a navigation target for the locomotion subsystem or for to answer user question. In detail, the symbolic knowledge base defines the meaning of task-relevant concepts, while the persistent perception layer provides their current physical realization in the environment. This decoupling makes the system more flexible, since the robot can reason about semantically meaningful targets even when their physical position changes over time.

Finally, we can state that the semantic layer therefore acts as the central interface between perception, interaction with humans, and embodied action. By combining persistent geometric information with symbolic relationships, it enables the robot to ground natural-language requests into specific physical entities and provides the spatial-semantic information required for object search and autonomous navigation.

4.4 Locomotion

The locomotion subsystem is responsible for converting a high-level navigation request into a sequence of motion commands that can be executed by the G1 humanoid robot. Its role is therefore to connect the semantic interaction layer with the low-level walking controller available in MJLab. The proposed solution follows a modular architecture in which semantic reasoning, path planning, waypoint tracking, and locomotion control are kept separated. This separation reduces dependencies between components and allows each module to be developed, tested, and used independently. Once the interaction subsystem, described in the following section, identifies a valid navigation target in the semantic representation of the environment, the locomotion subsystem determines how the robot can physically approach it. Instead of directly commanding the robot toward the geometric centre of the detected object, the system identifies a reachable navigation goal in the surrounding free space. This is necessary because the semantic position of an object, such as a desk or a bookshelf, may lie inside an occupied region and therefore cannot be used directly as a navigation goal. Similarly to how a person approaches a piece of furniture, the G1 is instead guided toward the closest available and reachable position around the target.

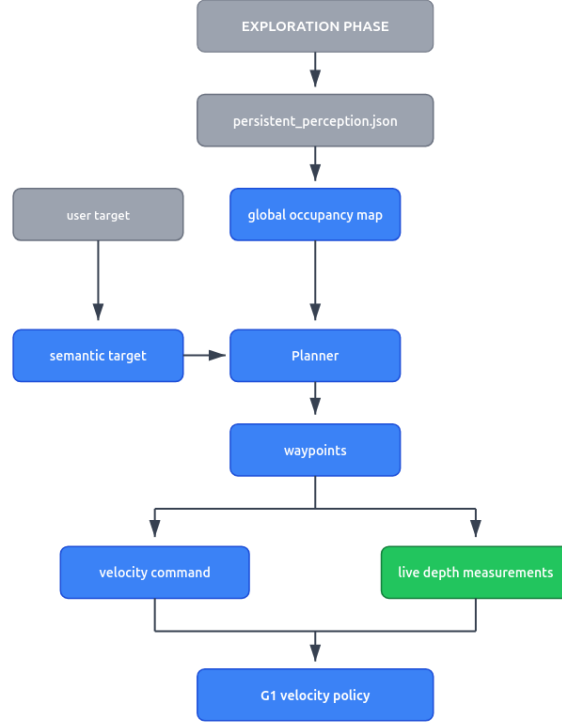


Figure 5: High-level locomotion pipeline from semantic target identification to robot motion execution.

To realize this architecture, several modules of the locomotion pipeline were developed specifically within the project, as illustrated in Fig. 6. These include occupancy-map generation, semantic goal selection, A* global path planning, path simplification, waypoint navigation, and the integration layer that connects the generated velocity references to the simulated G1. In contrast, the G1 robot model, the MJLab simulation framework, and the locomotion tracking velocity-tracking policy were integrated as existing foundational components.

Since the GAIT generation falls outside the main scope of the HRAI project, the locomotion policy is treated as an independent motion-execution component. Due to the available computational resources, the policy used in the project was trained for a limited number of training steps. Future developments could therefore adopt a more extensively trained policy or reuse an already available pre-trained controller. At the architectural level, the velocity-tracking policy remains fully decoupled from the higher-level reasoning and planning modules: it does not require information about the semantic meaning of the target or the structure of the complete trajectory. Instead, it receives only linear and angular velocity references, namely forward velocity, lateral velocity, and yaw rate, while mapping, path feasibility, and goal selection are handled by the higher-level components of the locomotion stack.



Figure 6: Locomotion subsystem components: in blue components developed within the project, in gray blocks the already existing components.

Following the standard navigation paradigms used in autonomous and mobile robotics, the locomotion subsystem adopts a 2D occupancy grid to represent traversable and obstructed regions of the simulated workspace. Rather than relying only on raw range sensing, the grid is generated from the persistent 3D spatial representation produced by the perception subsystem, providing a consistent metric representation for global path planning.

Once a semantic target has been identified, the navigation module determines the closest free and reachable position around it and plans in real time with the A* algorithm to compute a collision-free path from the robot’s current pose. Since the raw A* solution may contain a large number of intermediate grid cells, the resulting trajectory is simplified into a smaller sequence of meaningful waypoints. A closed-loop waypoint tracking controller then continuously evaluates the robot pose with respect to the next waypoint and converts the tracking error into translational and rotational velocity commands. Finally, these velocity references are finally executed by the G1 velocity-tracking policy integrated into MJLab. This hierarchical organisation separates semantic reasoning, global navigation, waypoint tracking, and low-level humanoid locomotion, creating a clear interface between Human-Robot Interaction and embodied motion execution. The resulting architecture therefore provides a modular and extensible baseline for autonomous semantic navigation within the simulated office environment.

4.5 Human-G1 interaction

The Human-Robot Interaction layer represents the high-level interface between the user, the semantic representation of the environment, and the locomotion subsystem. Its purpose is to transform a natural-language request into a task that can be grounded in the robot’s current knowledge of the environment and subsequently executed by the G1. Rather than associating user commands with predefined robot behaviours, the proposed architecture exploits the persistent semantic representation generated during

perception and exploration.

In the current experimental setup, interaction starts from a natural-language instruction provided through keyboard input. Based on the expected use cases of a Public Administration office, we assumed that typical requests will include asking the robot to locate an object or to navigate towards a task-relevant entity, such as a specific workstation. Each request is interpreted by an LLM that identifies both the intended action and a semantic target. Importantly, the language model does not generate geometric coordinates or directly control the robot. Instead, the semantic target is forwarded to the Semantic Query Engine, which combines the persistent perception map with the Semantic Relations Knowledge Base to identify the corresponding physical object. This organization introduces a clear separation between language understanding, semantic grounding, and physical execution. The interaction layer determines *what* the user is requesting, the semantic layer identifies *which physical entity* corresponds to that request, and the locomotion subsystem determines *how* the robot can reach it. Once a physical target has been resolved, it is forwarded to the navigation subsystem, which selects a reachable goal, computes the path, and generates the velocity references executed by the G1 locomotion policy in MJLab as shown in Fig. 7.

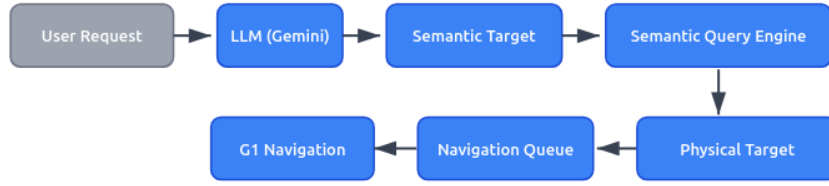


Figure 7: Interaction flow for a navigation task

Rather than allowing the LLM to freely infer arbitrary robot behaviours, the system proposed maps user requests onto a predefined set of supported intents, currently including *navigate*, *locate*, *describe*, and *unknown*. This design provides a controlled interface between natural-language understanding and robot execution, reducing the risk of generating actions that are not supported by the downstream modules while still preserving the flexibility of the LLM in interpreting different linguistic formulations. Following this approach, the intent set was designed to remain extensible according to the type of deployment and interaction required. Specifically, the introduction of the *describe* intent extends the system beyond spatial commands, allowing the robot to provide grounded information about perceived entities and details from the official Ministry of Finance and Treasury (MEF - DT) sources. The response strategy can also be adapted to the expected user profile: for example, an employee familiar with the environment may prefer short and direct responses, while an external visitor can

benefit from more descriptive explanations and guidance. This provides a first step towards a more context-aware and user-aware Human-Robot Interaction framework.

Finally, it is worth noting that one of the central design objectives was to keep the Human-Robot Interaction layer independent from the specific perception and locomotion implementations. The interaction logic operates on semantic entities rather than raw RGB-D data or object detections, while motion requests are expressed as task-level goals rather than joint-level commands. Existing components such as LOST-3DSG, MuJoCo, MJLab, and the G1 locomotion policy are therefore connected through the semantic and interaction modules developed within the project. This modularity makes it possible to replace or extend individual components, such as the Vision-Language Model, navigation planner, locomotion policy, or supported interaction intents, without redesigning the overall interaction architecture.

5 Implementation

As described in Section 4, the implementation follows a modular design in which each subsystem is responsible for a specific task and communicates with the others through ROS2. The project is built upon the MJLab framework, which provides the simulation environment, the reinforcement learning infrastructure, and the trained locomotion policy for the Unitree G1 humanoid robot. MuJoCo is used as the underlying physics engine, while the native MJLab `play.py` script is employed to execute the trained velocity-tracking policy. To avoid modifying the original MJLab source code a python virtual environment is used inside the Docker container to isolate project dependencies from those required by the original robotics framework. This simplifies package management while avoiding modifications to the base Docker image. Then the custom environment is integrated through a monkey-patching mechanism. This approach preserves compatibility with the original framework while allowing the trained locomotion policy to be executed inside an arbitrary custom environment. The software developed within this project is organized into five main directories:

- `exchange/environments/` contains the scene generation and simulation pipeline. A YAML description of the environment is converted into a MuJoCo XML scene through the `build_scene.py` script. The generated environment is then loaded into the MJLab simulation through `run_scene.py`, which applies the custom environment configuration and starts the execution of the trained velocity-tracking policy;
- `exchange/sensing/` contains the perception interface. A `CameraManager` acquires RGB images from a virtual camera mounted on the robot head. A dedicated ROS2 node periodically publishes these images, reproducing the communication interface expected on a physical robotic platform;
- `exchange/semantic_maps/` here the semantic perception and scene representation modules. A dedicated ROS2 node subscribes to the camera stream and processes the received frames independently from the locomotion controller. This node provides the integration point for LOST-3DSG, which will be used to perform open-vocabulary scene understanding and semantic tracking. The resulting information will then be integrated into the persistent semantic representation of the environment through the `ObjectManager` and the semantic mapping components;
- `exchange/locomotion/` here is handled how to navigate
- `exchange/interaction/` contains all the Python modules needed to interact with humans and transforms the received NL inputs in a set of actions or text to answer to the user request.

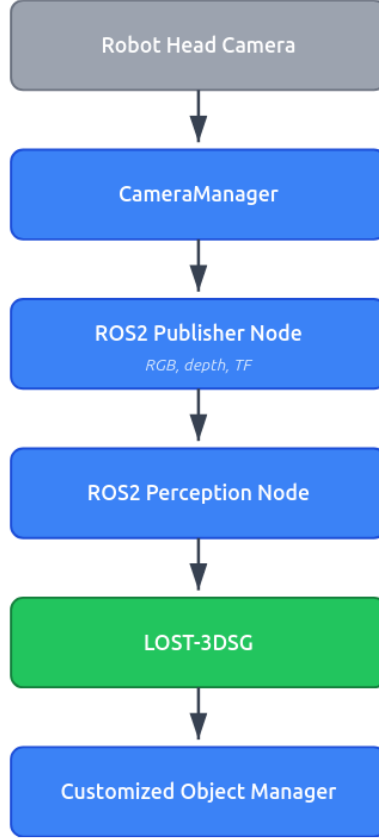


Figure 8: Perception pipeline

The interaction between the simulation and the perception pipeline relies on ROS2 topics. During the simulation, the camera publisher node acquires RGB images from the virtual robot-mounted camera and publishes them for the perception pipeline. The semantic perception node independently receives and processes incoming frames. This asynchronous execution model ensures that computationally intensive vision and semantic mapping algorithms do not directly interfere with the control frequency of the trained locomotion policy.

The next stage of the implementation will integrate LOST-3DSG into the semantic perception pipeline.

5.1 Scene Generation

The implementation of the virtual office is in charge of the `build_scene.py` which reads `scene.yaml`, creates the `jasset` and `jworldbody` sections, and generates a MuJoCo `jbody` for every object. Standard box objects are directly instantiated as MuJoCo box geometries, while mesh objects generate two independent geometries: a detailed mesh with `contype="0"` and `conaffinity="0"` for visualisation only, and an invisible box with active collision properties for physical interaction. Unfortunately, the external IKEA models introduced an additional side problem: the downloaded `.glb` files used

Draco mesh compression (KHR_draco_mesh_compression), which was not correctly supported by the Python mesh-processing tools used in the pipeline. This initially resulted in meshes containing vertices and faces but with zero spatial extent and zero surface area, ultimately causing MuJoCo to report mesh surface area is too small. Adopted solution is therefore to decode the Draco-compressed GLB before mesh optimisation, after which the geometry can be processed with tools such as trimesh and Open3D. The preprocessing stage performs optional quadratic-error mesh decimation to reduce complexity, followed by conservative removal of invalid, degenerate and duplicate triangles before exporting an OBJ suitable for MuJoCo. Importantly, the preprocessing is applied only to the visual representation; physical interaction remains delegated to simple collision primitives generated directly from the YAML dimensions.

5.2 LOST-3DSG integration

Compared to using a standalone object detector such as OWLv2, LOST-3DSG provides a broader framework for constructing lightweight open-vocabulary 3D scene graphs and maintaining semantic information over time. Therefore, the perception pipeline will not be limited to producing independent object detections for each frame. Instead, semantic observations collected over time will be associated and integrated into a persistent representation of the environment. Moreover, it is worth noting that adopting LOST-3DSG provides direct structural alignment with our local research, as the architecture originates from DIAG department. Beyond the benefit of methodological familiarity, this integration allows us to leverage an open-vocabulary 3D mapping pipeline that combines the contextual reasoning of VLMs with zero-shot local detectors and 3D geometric primitives. Consequently, our robot gains the spatial-semantic memory required to perform complex downstream tasks, such as object search and natural language navigation, without suffering from the frame-by-frame memory loss, duplicate spatial instances, or severe computational overhead typical of traditional dense 3D reconstruction methods.

5.2.1 Integration challenges and solutions applied

Spatial coordinates transformation During the initial integration phase, a synchronization bottleneck occurred within the perception pipeline due to the absence of a valid spatial transformation (TF) between the global coordinate system and the robot’s optical frame. The LOST-3DSG module inherently relies on time-synchronized RGB-D frames, intrinsic camera parameters, and the exact 3D pose of the sensor to project 2D bounding boxes into the global 3D space. To resolve this missing kinematic link, the custom ROS2 bridging node was modified to dynamically extract the camera’s global translation `cam_xpos` and rotation matrix `cam_xmat` directly from the MuJoCo physics engine state at each simulation step. The 3x3 rotation matrix was converted

into a normalized quaternion using MuJoCo’s native C-bindings, explicitly handling the structural mapping from MuJoCo’s scalar-first (w,x,y,z) to the ROS 2 standard vector-first (x,y,z,w) format. This data was then continuously broadcasted as a TransformStamped message via a tf2_ros TransformBroadcaster, successfully satisfying the pipeline’s spatial dependencies and unblocking the Vision-Language Model inference cycle.

VLM API revise The original framework implementation relied on gpt-5-mini via the OpenAI API for multimodal reasoning, our project constraints regarding API credits and the desire to evaluate a broader variety of architectures motivated a transition to Google’s Gemini models. This shift required modifying the perception module’s backend. Unfortunately, a critical compatibility issue emerged when attempting to use the standard OpenAI Python client as a wrapper for Google AI Studio endpoints. Specifically, encapsulating Base64-encoded images within the OpenAI-formatted JSON payload caused Google’s compatibility layer to fail, yielding misleading HTTP ‘404 error’ that falsely indicated the model was unsupported for content generation. To resolve this protocol mismatch, the codebase was refactored to bypass the OpenAI library entirely. A direct REST integration was implemented using Python’s native requests library, structuring the payload to strictly adhere to Google’s native API specifications, which successfully restored stable and accurate visual-language inference. Different models such as `gemini-2.5-flash` and `gemini-3.6-flash` has been tested and in the future a more in-depth evaluation could be made of the results achieved with the various models.

VLM output parsing In LOST-3DSG pipeline, a downstream Vision Language Model (VLM) call is performed on localized object crops to generate structured JSON metadata—capturing properties such as color, material, geometric shape, and semantic descriptions necessary to enrich 3D object instances. However, an issue arose during string deserialization, where standard JSON parsers (`json.loads`) raised syntax errors. This was caused by the new adopted VLM automatically wrapping its valid JSON payload in Markdown code blocks (*e.g.* “`json ...`”), which introduced non-JSON header and trailer tokens. We solved this issue by re-implementing the `process_crop_vlm()` function in order to correctly decode VLM’s answers. As future development to achieve a more robust and model-agnostic architecture across heterogeneous VLMs with unpredictable wrapping styles, future iterations should decouple raw string manipulation by integrating schema-constrained decoding alongside regex-based AST extraction with automatic fallback validation.

5.2.2 Architectural refactoring of `perception.py`

The original implementation of the perception pipeline executed all computationally intensive operations such as object detection (OWLv2), segmentation (ViT-SAM),

point-cloud generation, VLM-based attribute extraction, and 3D bounding-box computation, inside the ROS 2 executor thread. Because a single perception cycle takes approximately 6 seconds, while the orchestration timer fires every 0.5 seconds, the executor was permanently blocked causing a couple of critical failures:

- **Message backlog:** Messages about RGB, depth, and TF, accumulated in subscriber queues. When processing finally resumed, the node consumed frames that were 4–6 seconds old, leading to stale spatial data and duplicated objects in the downstream semantic graph.
- **TF lookup failures:** The transform buffer could not be updated in real time because `rclpy.spin()` was suspended. Consequently, `lookup_transform` requested timestamps that did not yet exist in the buffer, triggering "extrapolation into the future" errors!

As a solution, we introduced a producer–consumer pattern with two distinct threads in order to decouple the perception job: The timer callback (`_timer_tick`) was intentionally stripped of all blocking logic. It checks state predicates (`is_stationary`, `first_detection_done`, elapsed stationary time) and, when appropriate, enqueues the latest synchronized camera frame into a queue with unitary size. If the worker is still processing a previous frame, the old queued frame is discarded and replaced with the newest one. This guarantees that the worker always processes the most recent available data, eliminating temporal drift.

THREAD	RESPONSIBILITY	ROS 2 INTERACTION
Main (ROS) thread	<code>rclpy.spin()</code> , subscriber callbacks, lightweight timer tick, TF buffer maintenance, joint-state monitoring	Full access to ROS 2 API
Worker thread	Heavy perception logic: detection → segmentation → point-cloud filtering → parallel VLM calls → world-model update	Accesses cached camera data via a thread-safe <code>Queue</code>

Figure 9: Decoupled perception threads architecture

5.3 Object manager and semantic layer implementation

During the integration of LOST-3DSG semantic building process with the simulated G1 robot, the main issue was an unstable association between newly detected objects and the objects already stored in the persistent scene graph. This produced multiple duplicated nodes for the same physical object, especially for small objects such as laptops and lamps and after large viewpoint changes. After several tests we identify the

first source of error as the depth preprocessing inherited from the original TIAGo-oriented pipeline. The previous implementation assumed that large depth values were expressed in millimetres and automatically divided the whole depth image by 1000 whenever the maximum value exceeded a fixed threshold. This assumption was valid for depth streams such as 16UC1 commonly expressed in millimetres, but it was incorrect for the G1 MuJoCo simulation, where the 32FC1 depth image was already expressed in metres. Since the simulated camera could contain far-plane values above 20 m, the heuristic incorrectly converted meter values again, reducing object distances by a factor of 1000 and producing invalid 3D centroids and bounding boxes.

```
# LOST-3DSG PAPER VERSION SHAPED ON TIAGo robot
depth_array = np.asarray(depth_array).astype(float)
if depth_array.max() > 20.0:
    depth_array = depth_array / 1000.0
```

A second issue concerned the coordinate frames used during 3D reconstruction: RGB-D back-projection initially followed the optical convention, while the MuJoCo camera transform followed a different axis convention. This required an explicit conversion before transforming the reconstructed points into the global map frame. Finally, even after correcting the geometry, the original association logic still caused duplication because semantic similarity was evaluated as a mandatory gate before spatial consistency. As a consequence, a physically unchanged object could be rejected when the VLM generated a different description and the LOST similarity score dropped below the semantic threshold. The opposite situation was also observed for semantically equivalent labels, such as desk and table: the objects could have a very high overlap and semantic similarity but were rejected because their label strings were not identical.

Furthermore, percentile statistics of the depth values inside each segmentation mask were introduced during debugging. These logs showed that some masks contain two distinct depth clusters, for example the foreground lamp at approximately 3.5–4 m and background pixels at 8–9 m. This explained some remaining large bounding-box variations and showed that not every association failure originated from the scene update module.

Thus, the perception and association pipeline was progressively adapted to the sensing conventions of the G1 simulation. The most important change for persistent object association was the reordering of the matching conditions in `object_manager.py`. In the original logic, the LOST semantic similarity had to exceed `SIM.THRESHOLD` before IoU and centroid distance were even evaluated. Therefore, an object with nearly identical geometry but an unstable VLM description could be incorrectly classified as new.

The new logic gives priority to strong physical evidence: for objects with the same label, a sufficiently high 3D IoU is now enough to establish a match even when the

semantic similarity is low. The semantic score is retained for the more uncertain case in which the IoU is low but the centroids remain spatially close. This change successfully recovered cases such as a 'laptop' with a low semantic score but almost identical position and bounding box. A second conservative branch was added for semantic aliases: when the labels are different but both the LOST similarity and 3D overlap are very high, the system allows cross-label association. This solves cases such as 'table' versus 'desk', where the same physical furniture was described differently by the VLM. Experimental logs showed, for example, a 'table' observation matching an existing 'desk' with **lost_sim = 0.859**, **IoU = 0.811** and centroid distance of only **0.107 m**. At the same time, spatial constraints prevent two different instances of the same category from being merged, as observed for the two plants located several meters apart.

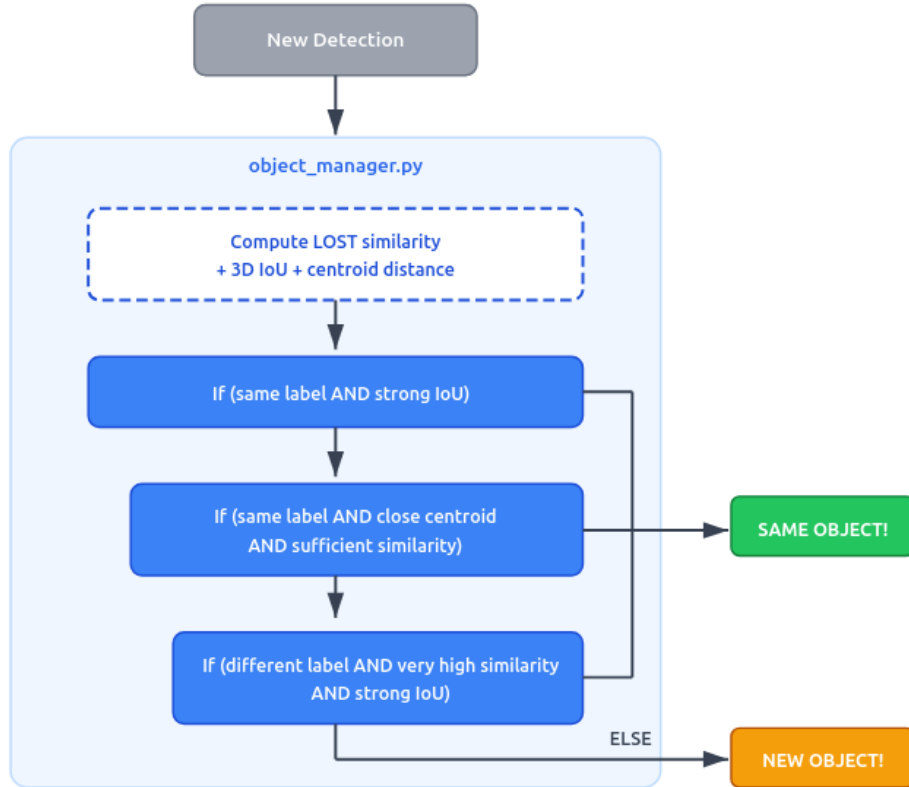


Figure 10: Flow diagram of the new decision order consisting in three nested if cases and a final else case to identify if we are dealing with a new object

However, the `persistent_perception.json` file, which contains all the perceived objects as detailed in [3], still stores some inconsistent representations. This occurs particularly when the open-vocabulary perception module assigns slightly different labels to the same object, as in 'mug' vs. 'cup' shown in Fig. 11. This happens due to a failure in retrieving attributes and descriptions from the VLM, leading to an unsuccessful cross-label association as discussed in the previous paragraph. This issue can be easily resolved by migrating to an on-premise VLM or switching to a pay-per-use

tier with higher RPM (Requests Per Minute) and TPM (Tokens Per Minute) limits.

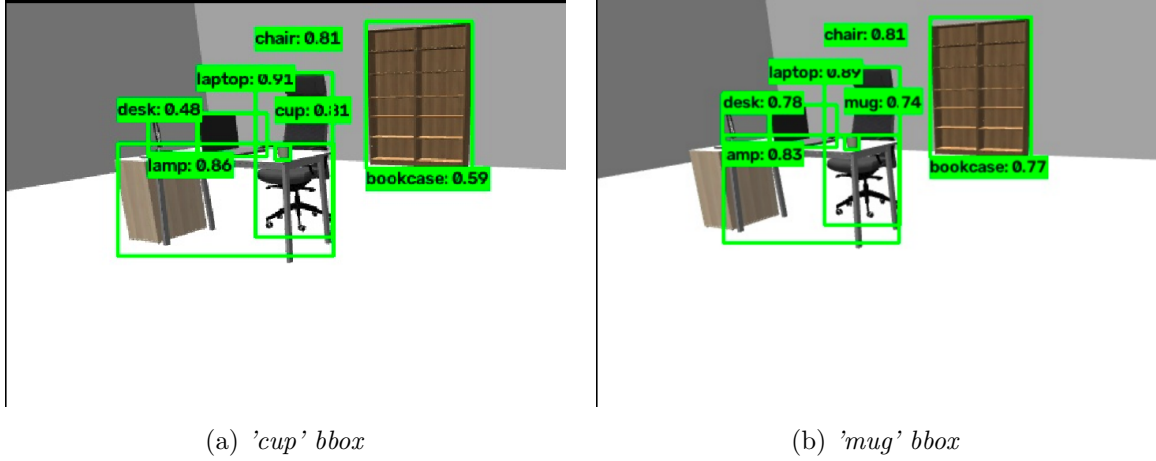


Figure 11: 'mug' vs 'cup' bounding box case for the same object seen in two different frames

5.3.1 Semantic map

The semantic layer was introduced to transform the raw output of the perception pipeline into a structured representation that can be directly queried by the higher-level reasoning components. The perception module detects objects in the environment, estimates their 3D bounding boxes from RGB-D observations, and associates semantic attributes such as label, color, material, shape, and natural-language description. These observations are progressively accumulated in the persistent world model, which represents the robot's current knowledge of the environment as defined in [3].

The persistent representation is then used to infer spatial relationships between objects. Instead of relying on predefined object coordinates, the semantic layer derives geometric information directly from the reconstructed 3D bounding boxes. Object centroids can therefore be computed at query time and used to identify local groups of semantically related objects. In the office scenario, this naturally produces geometric clusters corresponding to individual workstations, for example a desk located close to a chair, laptop, mug, or lamp. These spatial relationships make it possible to distinguish between otherwise visually similar objects. A desk can therefore be identified not only through its own properties, but also through the objects located on or near it.

To represent the scene knowledge without relying on a database or an unnecessarily complex data structure, we chose to encode inter-object relationships within a simple `SCENE_KB.yaml` file. This design opens the possibility for future developments where the humanoid robot can autonomously infer and add relations based on informative interactions with humans or its own exploration. Regarding the contents of this YAML file, it stores task-oriented semantic concepts and relations while keeping them decoupled from the continuously updated geometric world model. For instance, an entity such as *director_desk* can be defined through spatial relations, such as being

adjacent to a specific bookcase or supporting characteristic objects. The `semantic_engine.py` module then combines these symbolic constraints with the perceived objects from the persistent perception model, inherited from our implementation of the LOST-3DSG exploration pipeline, and retrieves the physical object that best satisfies the requested semantic target. This decoupling of semantic knowledge from perceived geometry is crucial: the knowledge base specifies what characterizes a target, whereas the perception module resolves where the corresponding physical object is currently located. The final persistent map intentionally preserves some uncertainty produced by the perception pipeline. In particular, objects may be geometrically localized even when semantic attribute extraction is incomplete. For instance, one detected laptop contains a valid 3D bounding box while its description, color, and material remain unknown. This represents a partial perception failure rather than a complete detection failure: the object can still participate in geometric reasoning even if its semantic characterization is unavailable. These cases highlight one of the limitations of the current system and show why semantic reasoning should not depend on a single perception attribute.

5.4 Locomotion implementation

The locomotion subsystem was implemented as a sequence of Python modules connecting the semantic representation of the environment to the velocity-tracking controller of the simulated G1 robot. The main components are the occupancy-map generator, the global planner, the waypoint navigation controller, and the MJLab integration layer. Their implementation is distributed across the modules `build_occupancy_map.py`, `planner.py`, `wp_navigation.py`, and `autonomous_runner.py`, respectively.

The navigation stack receives a physical target object resolved by the semantic layer rather than a manually specified Cartesian goal. The target geometry is then combined with the occupancy representation of the environment to identify a valid approach position, compute a collision-free path, and transform the resulting trajectory into velocity commands that can be followed by the G1 locomotion policy.

5.4.1 Occupancy map

Global navigation is performed on a two-dimensional occupancy grid generated from the persistent spatial representation produced by the perception subsystem. The implementation reads the reconstructed object instances and their 3D bounding boxes from the persistent perception map and projects their occupied regions onto the 2D XY plane. An occupancy map is then implemented using NumPy and stored as a compressed `.npz` file containing the binary occupancy grid, the map resolution, and the world-frame origin. Each grid cell is represented as either free or occupied. In the current implementation, a resolution of 0.10 m is used. Since the geometric bounding box of an obstacle describes only its physical dimensions, an additional safety margin

is applied during map construction according to the approximate footprint of the humanoid robot and an additional clearance margin. In the adopted configuration, a robot radius of 0.30 m and a safety margin of 0.15 m result in a total obstacle inflation of 0.45 m. This allows the global planner to reason about the clearance required by the robot rather than treating the G1 as a point mass.

The map-generation stage also performs a geometry-based filtering operation to avoid unnecessarily marking small objects placed on top of larger pieces of furniture as independent navigation obstacles. For example, as we can note in Fig.12, objects such as mugs, laptops, or lamps that are spatially contained within a desk footprint do not introduce additional occupied regions when the supporting desk already represents the relevant navigation obstacle.

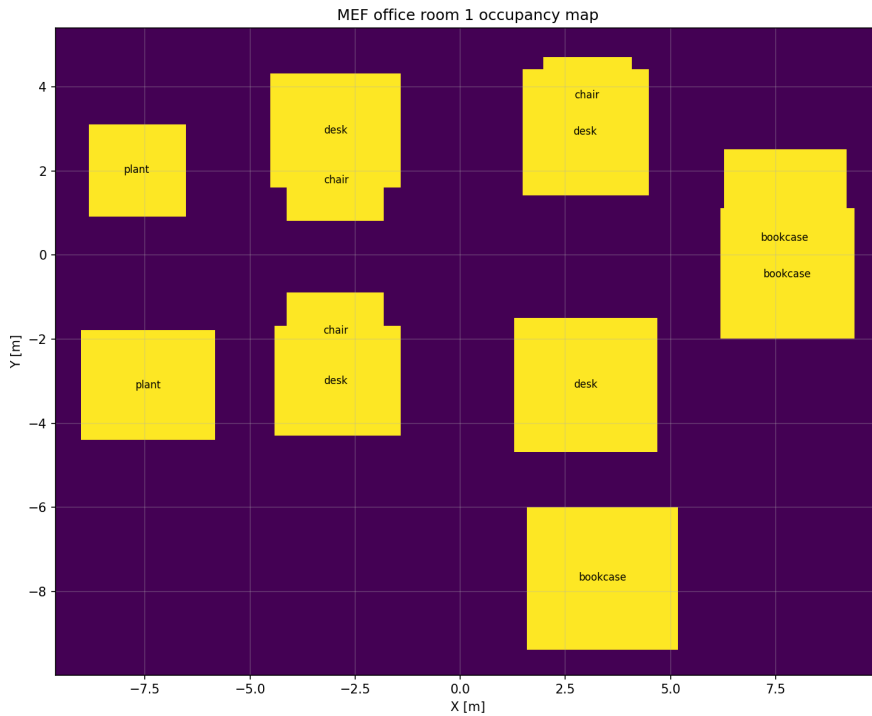


Figure 12: 2D occupancy map generated from the persistent 3D representation of the virtual MEF office. Occupied cells (in yellow) include the safety inflation margins for a safer navigation

5.4.2 Semantic goal selection and path planning

Path planning is implemented in `planner.py`. The planner operates directly on the binary occupancy grid and provides the transformations required to convert between world coordinates and discrete map cells. A semantic target cannot generally be used directly as the final navigation position. The centroid of an object such as a desk is usually located inside the corresponding occupied region. For this reason, the planner first identifies a nearby free-space location that can actually be reached by the robot. An initial implementation generated a limited number of predefined candidate points

around the target bounding box. However, this strategy was strongly dependent on the object dimensions and could fail whenever the generated candidates were located inside inflated obstacles or disconnected regions of free space. The final implementation therefore adopts a reachability-based goal-selection strategy.

Starting from the robot’s current grid cell, a flood-fill exploration is performed over the free-space component that is reachable by the robot. The exploration follows the same neighbourhood constraints used by the global planner. Among all reachable cells within a predefined search radius of the semantic target, the cell whose centre has the minimum Euclidean distance from the target is selected as the navigation goal. Formally, given the set of cells $\mathcal{R}$ reachable from the current robot position and the target position $\mathbf{p}_t$, the navigation goal is selected as

$$c^* = \arg \min_{c \in \mathcal{R}} \|\mathbf{p}(c) - \mathbf{p}_t\|_2 \quad (1)$$

where $\mathbf{p}(c)$ represents the world coordinates of the centre of cell c . Only cells within the configured maximum search radius are considered. In that way, we can guarantee that the selected goal is not only geometrically free but also belongs to the connected free-space region that can be reached from the current robot pose.

Once the goal has been selected, the global path is computed through an A* search. The implementation uses an 8-connected grid, allowing horizontal, vertical, and diagonal movements. Straight transitions have unitary cost, while diagonal transitions have a cost of $\sqrt{2}$. An octile-distance heuristic is used consistently with this motion model. In addition, diagonal transitions that would pass between two occupied neighbouring cells are rejected to prevent corner cutting.

The resulting A* path consists of a potentially large number of adjacent grid cells. A path-simplification stage therefore removes unnecessary intermediate points whenever a direct collision-free connection exists between non-consecutive path nodes. The simplified path is finally converted back into world coordinates and used as a sequence of navigation waypoints.

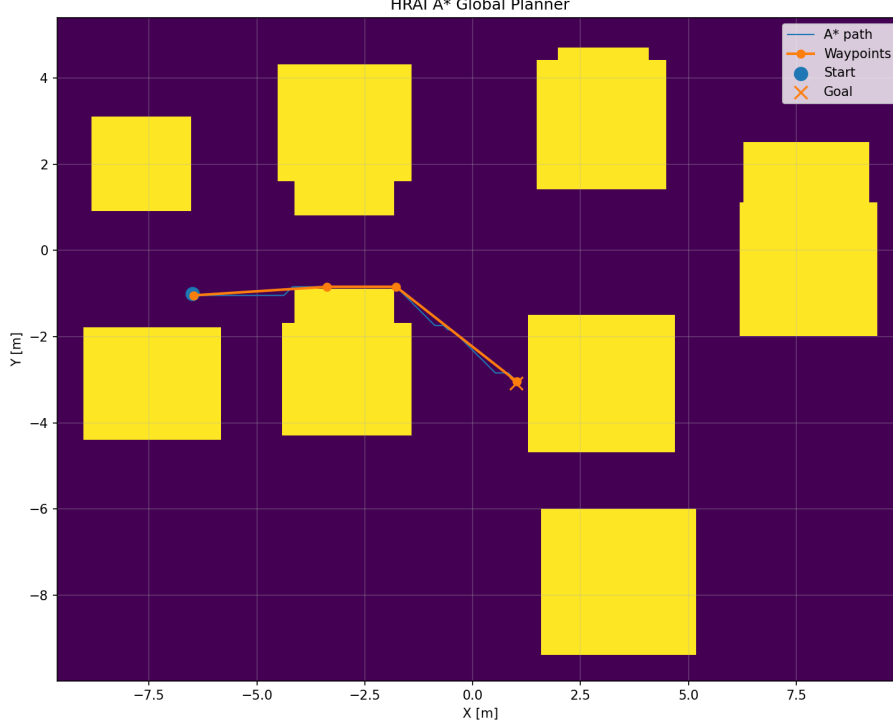


Figure 13: Example of semantic goal selection and global planning. The semantic target may lie inside an occupied region; the nearest reachable free-space goal is selected and connected to the robot through an A* path, which is subsequently simplified into navigation waypoints.

5.4.3 Waypoint navigation

The simplified global trajectory is executed through the closed-loop controller implemented in `wp_navigation.py`. The `WaypointFollower` class sequentially tracks each waypoint using the current position and orientation of the robot as feedback. For a waypoint located at (x_w, y_w) and a robot pose (x_r, y_r, ψ_r) , the position error is first evaluated in the world frame and then transformed into the robot local reference frame:

$$\begin{bmatrix} e_x^r \\ e_y^r \end{bmatrix} = \begin{bmatrix} \cos \psi_r & \sin \psi_r \\ -\sin \psi_r & \cos \psi_r \end{bmatrix} \begin{bmatrix} x_w - x_r \\ y_w - y_r \end{bmatrix} \quad (2)$$

Local tracking error is then converted into forward and lateral velocity references, while the angular difference between the robot heading and the direction of the waypoint is used to generate the yaw-rate command. The controller produces the command vector

$$\mathbf{v}_{cmd} = \begin{bmatrix} v_x & v_y & \omega_z \end{bmatrix}^T \quad (3)$$

These velocity values are saturated according to limits compatible with the locomotion controller. In the current configuration, we empirically set the maximum forward velocity to 0.40 m/s, the maximum lateral velocity to 0.20 m/s, and the maximum yaw rate is 0.50 rad/s. A waypoint is considered reached when the robot enters a positional tolerance of 0.20 m, after which the controller automatically switches

to the following waypoint. This feedback mechanism is particularly important for humanoid locomotion, since the actual robot trajectory does not exactly coincide with the geometric trajectory generated by the grid planner. The controller therefore continuously recomputes the command using the measured G1 pose rather than executing a predefined open-loop sequence of velocities.

5.4.4 Walking in MjLab

The connection between global navigation and low-level humanoid locomotion has been implemented in `autonomous_runner.py` sketch. The runner acts as the interface between the navigation modules and the MjLab simulation environment.

The simulation is based on the Unitree G1 model from the official Unitree’s GitHub repository and on the velocity-tracking task `Mjlab-Velocity-Flat-Unitree-G1`. A trained locomotion policy receives the three-dimensional velocity command (v_x, v_y, ω_z) and converts it into the low-level actions required to control the robot joints.

During each simulation iteration, the runner extracts the current position and orientation of the G1 base directly from the MuJoCo state, then the robot pose is passed to the `WaypointFollower`, which generates the next velocity reference. These values are subsequently written into the active MjLab `twist` command and work as a input for the velocity-tracking policy. This process is repeated at every control step until the current waypoint is reached or until the complete navigation path has been executed. The locomotion runner is also connected to the higher-level Human-Robot Interaction pipeline through a thread-safe request queue. The interaction and semantic modules, as we are going to see in the next section, resolve the user’s natural-language request into a physical target object and enqueue the corresponding navigation request. When a navigation request becomes available, `autonomous_runner.py` retrieves the target, invokes the planner, initialises the waypoint controller, and starts feeding velocity references to the G1 controller. Finally, resulting implementation therefore preserves a clear interface between the different control levels. *Semantic layer* specifies the physical object to be reached, *the planner* determines a valid geometric path, *controller* generates velocity references, and the existing MjLab locomotion policy converts these references into embodied walking behaviour.

5.5 G1-Humans interaction

The purpose of this section is to receive a natural-language request from the user, interpret the requested task, ground it in the semantic representation of the environment, and forward the resulting command to the appropriate downstream component. The interaction pipeline is mainly implemented through `HG1AI_interaction.py`, `task_parser.py`, and the semantic query interface described in the previous section.

A key design choice was to keep natural language understanding separated from

geometric reasoning. The language model is not asked to estimate coordinates or directly control the robot. Instead, it translates the user’s request into a structured semantic task and the resulting semantic target is later resolved against the robot’s persistent representation of the environment, while motion planning remains entirely managed by the locomotion subsystem.

5.5.1 Natural Language task interpretation

Natural language interpretation is implemented through Google’s Gemini API. In the current implementation, `gemini-3.6-flash` is accessed through a direct REST interface using the Python `requests` library. A direct REST integration was preferred in order to maintain explicit control over the request format and to use the same structure adopted in the perception subsystem. At runtime, the prompt, stored in `task_prompt.txt`, is completed with the semantic targets currently available to the robot and with the user’s request. Gemini is instructed to map the request into a restricted set of supported intents, currently including *navigate*, *locate*, *describe* and *unknown*, and to associate the request with one of the available semantic target identifiers. For instance, as we can see in Fig. 14, a request such as *‘Could you show me the path to the director’s desk?’* is translated into the semantic task *navigate* associated with the identifier `director_desk`.

Using this kind of constrained structured response simplifies the interface between the probabilistic language model and the deterministic components of the robotic system. The parser expects a JSON structure containing the interpreted intent and semantic target and validates the returned fields before forwarding the result to the remaining modules. The language model is explicitly prevented from inventing targets that are not available in the semantic representation by only identifying the semantic entity referred to by the user not its position. This separation provides an important safety and architectural boundary: natural-language reasoning is used to understand *what* the user wants, while the robot’s world model and navigation stack determine *where* the requested entity is located and *how* it can be reached.

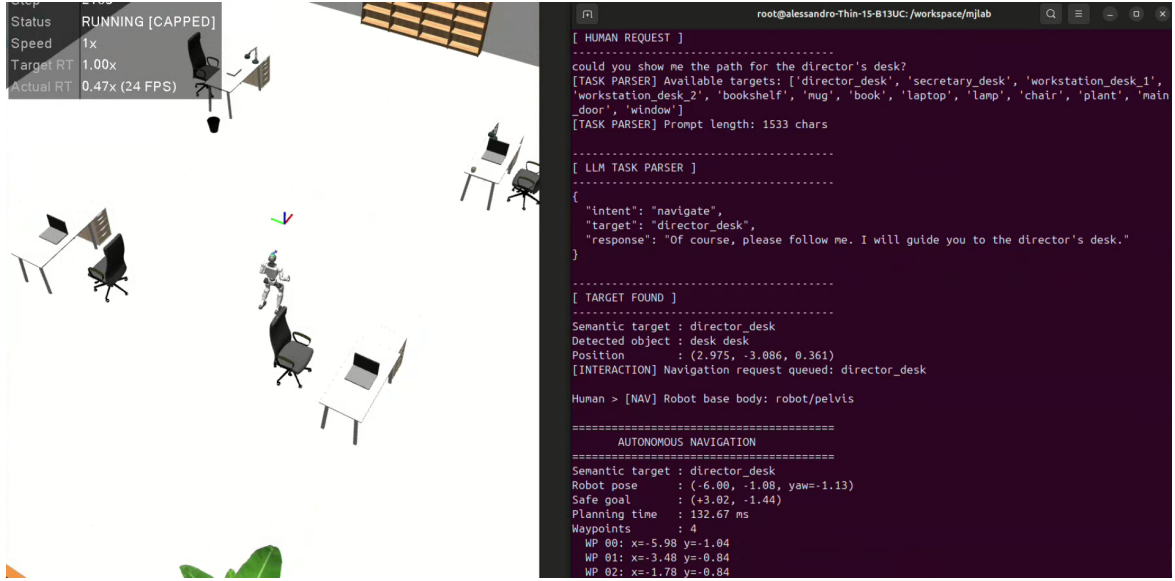


Figure 14: An instance of navigation task. On the right side we can appreciate all the steps performed by the system as described in 'Solution' section, and the LLM immediate reply

5.5.2 Semantic grounding and task answer

Once a structured task has been obtained, the semantic target returned by Gemini is passed to the **Semantic Query Engine**, which combines the semantic relations Knowledge Base with the persistent perception representation. The query engine retrieves the physical object that satisfies the semantic constraints associated with the requested concept and finally an abstract target such as '**director_desk**' is converted into a specific perceived object containing the current 3D bounding box, semantic attributes, and spatial information required by downstream components. We can consider this process as a two-stage grounding procedure:

1. **Language grounding**: the user request is mapped to a semantic target identifier by Gemini.
2. **Spatial grounding**: the semantic target is resolved into a physical object through the Semantic Query Engine and the persistent world representation.

This distinction is particularly important because it prevents the language model from becoming directly dependent on the geometric representation of the simulated environment. Changes in object position or perception results therefore do not require changes to the language interpretation layer.

Anyway, we have to note that at the current stage, the action performed after semantic grounding depends on the intent identified by the task parser. A *locate* request is handled directly using the spatial information returned by the semantic layer, while a *navigate* request forwards the resolved physical object to the navigation subsystem. The *describe* intent follows a different interaction path: after the requested semantic

target has been grounded to one or more physical objects, the system collects the available grounded information associated with those entities, including semantic attributes, spatial relations, and object properties. This context is then passed to a second Gemini call dedicated to natural language response generation. The response generator is instructed to produce a clear and descriptive answer for an external visitor while relying only on the information retrieved from the semantic representation, avoiding the introduction of unsupported facts. This two-stage LLM architecture separates task interpretation from response generation. The first Gemini call determines the user’s intent and semantic target, whereas the second call is used only when a grounded natural-language description is required. In this way, the final answer remains anchored to the robot’s internal representation of the environment rather than to the general knowledge of the language model. If the requested target cannot be interpreted or successfully grounded, in order to prevent the spread of misleading and not true information, no navigation command or descriptive response based on unverified information is generated.

```
Human > can you locate the director's mug?

-----
[ HUMAN REQUEST ]
-----
can you locate the director's mug?
[TASK PARSER] Available targets: ['director_desk', 'secretary_desk', 'workstation_desk_1', 'workstation_desk_2', 'bookshelf', 'mug', 'book', 'laptop', 'lamp', 'chair', 'plant', 'main_door', 'window']
[TASK PARSER] Prompt length: 1202 chars

-----
[ LLM TASK PARSER ]
-----
{
  "intent": "locate",
  "target": "mug",
  "response": "I have identified the location of the mug."
}

-----
[ TARGET FOUND ]
-----
Semantic target : mug
Detected object : mug
Position       : (2.368, -2.780, 0.801)

Human > 
```

Figure 15: Two-stage grounding process for the 'director's mug' object. Natural language is first mapped to a semantic target and subsequently resolved into a physical object using the semantic representation of the environment.

5.5.3 Runtime integration

A final implementation challenge concerned the concurrent execution between the previously described Human-Robot Interaction pipeline and the MJLab simulation. The MJLab play loop is blocking and the simulator, including its graphical viewer, is expected to remain in the main application thread. Executing the textual interaction

loop sequentially in the same thread would therefore prevent the robot simulation from progressing while waiting for user input.

To address this issue, the complete application was organised as a single Python process containing two execution threads. The main thread is dedicated to the MJLab simulation and G1 control, while a background thread manages user input, LLM (Gemini) task parsing, and semantic grounding. Once the interaction thread resolves a valid navigation target, the corresponding physical object is inserted into the navigation queue to make it visible to the `AutonomousSceneRunner`, which periodically checks the queue and starts the navigation procedure when a new request becomes available. The complete HRAI application is launched through `HG1AI_interaction.py`, which acts as the single entry point of the system. At startup, it initialises the interaction components and the autonomous locomotion runner, creates the shared navigation queue, launches the interaction logic in a background thread, and finally starts the MJLab simulation in the main thread. In conclusion, the proposed implementation establishes a clear boundary between language reasoning, semantic grounding, and physical execution.. Gemini, or whichever LLM model, interprets the user’s intention without controlling the robot directly; the semantic layer resolves the requested concept against the perceived environment; and the locomotion subsystem independently determines and executes a feasible path. At the same time, the asynchronous runtime organisation allows interaction to remain available while the simulated robot is operating, providing the basis for a continuous HRAI loop.

6 Experimental Evaluation

Following the benchmarking principles adopted by the European Robotics League [4], our experimental evaluation was designed to assess the system through repeated task executions under controlled variations rather than through a single successful demonstration. In particular, the ERL methodology emphasises the definition of well-specified benchmark questions, the controlled variation of relevant scenario features, the use of measurable performance indicators, and the analysis of multiple experimental instantiations. Trying to follow this methodology as much as possible, three representative interaction tasks were considered: *navigate*, *locate*, and *describe*. Each task was executed five times, varying the natural-language formulation and the semantic target while keeping the overall environment unchanged. This allows the evaluation to consider both task completion, both the robustness of language interpretation, semantic grounding, and downstream execution. Failures were additionally associated with the stage in which they occurred, such as task parsing, semantic grounding, path planning, locomotion, or response generation.

Score	Evaluation criterion
1.0	Intent correctly identified, semantic grounding correct and task completely executed with an output fully supported by the available world representation
0.75	Intent correctly identified and task substantially successful, but with a minor semantic mismatch, incomplete information or limited deviation from the requested goal
0.5	Intent correctly identified, but the task is only partially completed due to grounding, planning, execution, or response limitations
0.0	Complete task failure due to incorrect intent, failed grounding, unsupported response or unknown reason

Table 1: General scoring criteria adopted for the experimental evaluation

6.1 Navigation task evaluation

The *navigate* benchmark evaluates the complete pipeline from the human input to physical motion execution. Each trial starts from a user request expressed with a different linguistic formulation and referring to a different semantic target. A trial is considered fully successful when the task parser correctly identifies the *navigate* intent, the semantic target is correctly grounded to a physical object, a valid path is generated and the G1 reaches the selected navigation goal.

The evaluation also records the stage responsible for eventual failures. During repeated tests, one recurrent planning limitation was observed when the robot’s current pose was mapped to a grid cell marked as occupied as described in Fig.. In these cases, semantic interpretation and grounding were correct, but the navigation pipeline stopped before A* execution with a *start grid position occupied* error. This behaviour is related to the conservative obstacle inflation adopted in the occupancy map and to discretisation effects near furniture or other obstacles. These kind of trials are not classified as semantic failures, but as planning-level failures. A future improvement could be the adoption of use a more adaptive and less conservative local representation of the space.

Test	User request	Outcome	Score
1	"can you guide me to the bookshelf?"	Intent correct, target grounded, path generated, goal reached	1
2	"Hey, let's go to the director's desk"	Intent correct, target grounded, path generated, goal reached	1
3	"Guide me to the workstation"	Intent correct, target grounded, path generated, goal reached	1
4	"Let's move to the office entrance"	Intent and grounding correct, planning failed because start cell was occupied	0.5
5	"Can you guide me to the mug?"	Intent correct, target grounded, path generated, goal reached	1

Table 2: Experimental results for the navigation task.

6.2 Location task evaluation

The *locate* benchmark evaluates the ability of the system to interpret a spatial-information request and correctly associate the requested semantic concept with an object in the persistent world representation. Unlike the navigation task, no physical motion is required. The evaluation therefore focuses on intent recognition, semantic target identification, successful grounding, and consistency of the returned spatial information.

For this kind of task a test is considered fully successful when the requested entity is correctly interpreted and resolved to the expected physical object. Partial scores are assigned when the intent is correct but the target remains ambiguous or when multiple candidate instances cannot be uniquely distinguished. This is particularly relevant for categories such as *chair* or *plant*, for which multiple physical instances may exist in the environment. Such cases highlight the limitation of selecting a single grounded instance without an explicit clarification dialogue with the human user.

Test	User request	Outcome	Score
1	"Secretary desk"	Intent correct, target grounded to expected object, valid position returned	1
2	"Director's desk"	Intent correct, target grounded to expected object, valid position returned	1
3	"mug"	Intent correct, target grounded to expected object, valid position returned	1
4	"cup"	Intent correct, target grounded to expected object, valid position returned	1
5	"bookshelf"	Intent correct, targets grounded to expected objects, valid positions returned	1

Table 3: Experimental results for the location task.

6.3 Description task evaluation

The *describe* benchmark evaluates the ability of the interaction subsystem to interpret an information request, ground it to the semantic representation of the environment, and generate a natural language answer supported by the available knowledge. A trial receives full score when the requested entity is correctly identified and the generated response is both relevant to the user request and fully grounded in the information stored in the semantic world model and the Semantic Relations Knowledge Base. Particular attention was given to hallucination avoidance. For example, when the user

asked for information about the department director, the system correctly identified the *describe* intent but mapped the request to the closest available semantic entity, namely the *director_desk*. The generated answer explicitly stated that personal information about the director was not available and then provided a grounded description of the corresponding workstation. This behaviour was considered partially successful because the system avoided unsupported claims and remained grounded also if the semantic target did not fully correspond to the person requested by the user. Hence, according to the adopted scoring scheme, this trial was therefore assigned a partial score rather than full success.

Test	User request	Outcome	Score
1	"Tell if there is any window"	Intent correct, no target identified	0.5
2	"Describe the Lamp"	Intent correct, multiple targets identified, semantic match	1
3	"Can you tell me more about the director of the department?"	Intent correct, target grounded to the director's desk, Response remained grounded and explicitly stated that personal information about the director was unavailable. Semantic match only partial.	0.75
4	"Describe the plants"	Intent correct, multiple targets identified, semantic match	1
5	is there any desk with a lamp? describe the lamp	Intent correct, multiple targets identified, semantic match	1

Table 4: Experimental results for the description task.

In conclusions, it's worth noting that given the limited number of repetitions, the reported results should be interpreted as a functional validation of the proposed

architecture rather than as a statistically significant performance benchmark.

7 Results

Results are presented from two complementary perspectives. First, the internal representation built by the robot during the exploration phase is analysed, showing how raw visual observations are transformed into a persistent semantic representation of the office. Second, the final interaction behaviour is presented through representative examples involving semantic reasoning, natural-language description, object localisation, and navigation.

7.1 Exploration results

During the exploration phase, the semantic mapping pipeline was progressively validated through a series of increasingly complex experiments. Initial tests were performed in simplified scenes with the robot kept stationary, in order to verify object detection, segmentation, depth-based localization, and the construction of the persistent representation under controlled conditions. The setup was then gradually extended to the complete MEF office, where the robot was manually teleoperated through the environment to safely explore the whole room and observe the same objects from multiple viewpoints. One of the main challenges encountered during this process was maintaining a consistent representation across successive observations. In particular, the system had to distinguish between repeated detections of the same physical object and genuinely different instances of visually identical objects, such as the IKEA chairs, located at different positions. Avoiding duplicate entities while preserving distinct instances therefore represented one of the most significant achievements of the perception and mapping pipeline. The final system is able to associate repeated observations of the same object with a persistent entity while keeping separate objects of the same semantic class when their spatial evidence indicates that they correspond to different physical instances. Fig. 16

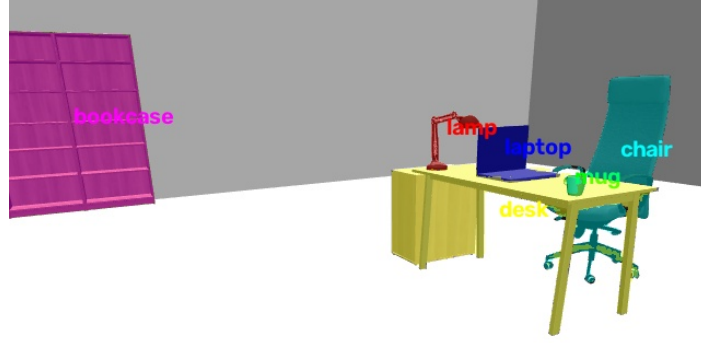


Figure 16: Perception output during office exploration. Segmentation masks are overlaid on the RGB observations to show the objects detected before their integration into the persistent semantic representation.

Relying on the collected data, the robot incrementally constructs a semantic graph representing its internal knowledge of the simulated MEF office. Each node corresponds to the position in XYZ space of an identified object. To assess the capabilities and limitations of the Object Manager and the overall perception pipeline, the exploration was performed on virtual environments of progressively increasing complexity. The initial scenario consisted of a single desk station containing only a 'desk', a 'notebook', 'lamp', and a 'chair'. Additional elements, such as plants, were then introduced, followed by further desk stations and objects, resulting in a more complex office environment. This incremental approach allowed us to observe how the perception and semantic mapping pipeline behaved as the number of objects and potential spatial relationships increased. In Fig. 17 the final semantic graph obtained after the exploration of the complete simulated office environment. The resulting graph represents the robot's internal understanding of its surroundings and provides the connection between low-level visual perception and the higher-level reasoning performed by the LLM/VLM-based components.

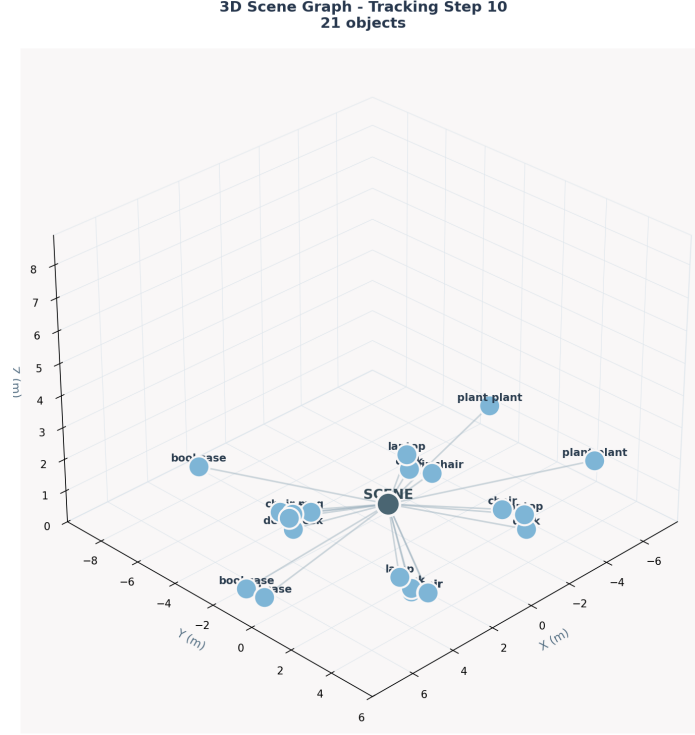


Figure 17: G1’s internal representation of the explored environment. We can easily recognize multiple instances of similar object classes. In particular, the representation contains the four main desks used in the office scenario together with multiple bookshelves, plants and other objects located on the workstations.

7.2 Interaction and task execution results

The interaction experiments show how the same semantic representation can support different behaviours depending on the user request. Instead of mapping directly the commands expressed in natural language into robot motion inputs, the system first interpret the request at semantic level, ground against the internal world representation, and only then processed according to the identified intent. This approach lead to a more ‘conservative’ and grounded behaviour as we can see from the Experimental evaluation results.

7.2.1 Reasoning over multiple object instances

An important behaviour observed during the experiments concerns semantic targets associated with multiple physical instances in the environment. The persistent representation contains, as will be in reality, multiple lamps and plants. When the user asks for a description of such a semantic category, the system is designed to do not collapse the results into a single perceived object. Instead, all grounded instances are retained and included in the context passed to the LLM response generator. This solution is actually implemented for both *location* and *describe* task and has returned the expected results

for all the objects which have multiple instances such as 'lamp' and 'plant' as proved in Fig.18. This behaviour demonstrates that the interaction layer is able to reason over sets of perceived entities rather than assuming a one-to-one correspondence between a semantic label and a physical object.

```

-----
[ TARGET FOUND ]
-----
Semantic target : plant
Detected objects: 2 instances
[1] plant plant @ (-7.704, 2.031, 0.762)
[2] plant plant @ (-7.429, -3.094, 1.025)

G1 > Hello! I found 2 plants in this area, and they are quite different from each other.

The first one is an artificial plant made of plastic, sitting in a simple brown pot. It has
fine, feathery foliage with delicate slender fronds rising upward, creating a light and ai
ry canopy.

The second one is a real, organic plant, also in a simple brown pot. This one features nume
rous large, broad leaves with prominent central veins, forming a lush and dense cluster wit
h a mix of rich bright and dark green shades.

```

Figure 18: Examples of grounded descriptions involving multiple perceived instances. The system recognises multiple plants in the semantic world model and generates the response using all the corresponding grounded entities.

7.2.2 Answering under incomplete semantic knowledge

A second representative interaction concerns the request: *“Can you tell me more about the director of the department?”*. The semantic knowledge base does not contain a representation of the director as a person. However, it contains the task-relevant semantic entity `director_desk`. Then the task parser autonomously associates the request with the closest available semantic target and rather than inventing personal information that is not contained in the internal representation, the LLM explicitly states that such information is not available and provides only grounded information concerning the director’s workstation. It then suggests consulting an official institutional source for further information.

Although this interaction does not represent a perfect semantic match between the requested person and the available target, it demonstrates an important reasoning behaviour: the system preserves the distinction between known and unknown information and avoids generating unsupported factual claims.

```

-----
[ TARGET FOUND ]
-----
Semantic target : director_desk
Detected object : desk desk
Position       : (2.975, -3.086, 0.361)

G1 > Welcome to the Ministry of Economy and Finance! If you are looking for the Director
's office, you can identify their main workspace by looking for a large rectangular wood
en workstation with a smooth white top and sleek metal legs. It is currently set up with
a laptop, a desk lamp, and a mug on top, with an office chair right beside it.

If you need further official information regarding the Department of the Treasury's orga
nization, leadership, or services, feel free to visit the official portal at https://www
.dt.mef.gov.it/it/index.html. Please let me know if you need help finding your way there
!

```

Figure 19: Interaction involving information not explicitly represented in the semantic world model. The system avoids generating unsupported information about the department director and instead provides grounded information associated with the available semantic target.

7.2.3 Navigation behaviour

To conclude the discussion on tasks and results, navigation requests demonstrate the integration between natural-language understanding, semantic grounding, global planning, and locomotion execution. Once an unambiguous physical target is identified, the corresponding geometric information is passed to the navigation subsystem, which selects a reachable goal, generates a path through the occupancy map, simplifies it into waypoints, and commands the G1 through the velocity-tracking locomotion policy. An additional behaviour emerges when multiple physical objects satisfy the same semantic target. For example, a request such as *“Take me to the lamp”* may correspond to more than one valid lamp in the semantic world model. In this case, the system instead of arbitrarily selecting one destination, inform the user that the request is ambiguous asking to formulate it again. As future extension we could have an ‘upgrade’ of the grounding process in order to reduce the ambiguous cases and improve the interaction with the user. In any case, the system will continue to have a different behaviour depending of the situation.

7.3 Video demonstrations

The accompanying material includes two demonstration videos, each containing multiple interaction tasks executed in sequence. Rather than showing isolated examples, the videos capture the behaviour of the complete HRAI pipeline over consecutive requests. For instance, a single recording may include two navigation tasks followed by a descriptive interaction, making it possible to observe how the system transitions between different intents while maintaining the same semantic world representation.

The videos are particularly useful for assessing aspects that are difficult to capture through static screenshots, such as inference time, end-to-end latency, and the temporal coordination between interaction and simulation. In particular, the response time

of the Gemini APIs is not constant across different requests. Some interactions are processed almost immediately, while others introduce a delay in the pipeline. We expect that this variability is mainly associated with the dependence on external cloud services, whose latency can change according to network conditions, service load, and API usage constraints.

Despite this variability, the observed waiting times remain acceptable for the considered experimental setup and do not prevent the execution of the interaction tasks. Nevertheless, for a real deployment, this dependency could be reduced by adopting a paid service tier with more predictable quotas and availability, or, more significantly, by moving the LLM components to an on-premise inference infrastructure. The latter solution would also provide greater control over latency, service continuity, and data handling.

8 Conclusions

One of the main lessons learned during the development is that the most challenging part of an embodied AI system is not necessarily the implementation of the individual components, but their consistent integration. Perception, semantic reasoning, language understanding, planning, and locomotion operate on different representations and at different time scales, and reliable interaction requires these representations to remain coherent throughout the complete pipeline.

A second important lesson concerns persistent perception: repeatedly detecting an object is considerably simpler than maintaining a consistent representation of that object over time and across viewpoints. A semantic map becomes useful for interaction only when repeated observations of the same physical entity can be associated correctly, while different instances of the same semantic class remain distinguishable.

Finally, one of the most important lessons I learned from this project is knowing when to stop. As the length of this report probably suggests, it is easy to keep extending a project when the topics are interesting and the available tools are evolving almost as quickly as they are being used. There were many additional features that I would have liked to include, as well as several possible ways to further refine and optimise the proposed solutions.

In some cases, these improvements led to architectural changes that proved particularly valuable. One example was the decoupling of the ROS2 listener workload from the processing pipeline by using multiple threads, a strategy that was later adopted again to separate the interaction logic from the simulation loop. However, continuous refinement also carries the risk of making the development process open-ended. At some point, it becomes necessary to stop adding features, assess what has actually been achieved, understand which design choices worked and which limitations remain, and use this evaluation to define the next meaningful steps.

From this perspective, learning to balance technical ambition with development boundaries was itself the most valuable lessons of the entire project.

8.1 Challenges

Some parts of the project, as expected, proved more difficult than initially prevented. The first was persistent object association during environment exploration as stated in Implementation section. When the robot observes the same area from multiple viewpoints, visually similar detections must be associated with previously observed entities without incorrectly merging physically distinct instances. Achieving a stable representation while preserving multiple instances of objects such as lamps, plants, desks, and chairs required a careful combination of semantic and geometric information. A second major challenge was the integration of semantic targets with navigation.

Semantic objects are defined by their perceived geometry, while the navigation system requires a valid and reachable goal in free space. Therefore, the centre of a detected object cannot generally be used directly as a navigation target. The implemented solution selects a reachable free position close to the semantic target and then plans a path using the occupancy representation.

The experimental evaluation also revealed limitations related to grid discretisation and obstacle inflation. In some cases, after moving through the environment, the current robot position was mapped to a cell classified as occupied, preventing the planner from starting a new navigation task. This behaviour does not originate from language understanding or semantic grounding, but from the current global planning representation and its conservative treatment of obstacles.

Finally, ambiguity in natural-language references remains a relevant HRI challenge. When multiple physical instances satisfy the same semantic target, descriptive and location requests can preserve and return all valid instances. Navigation, however, requires a unique physical destination. The current system detects this ambiguity and avoids selecting an arbitrary target, but it does not yet support a complete clarification dialogue capable of resolving a subsequent user specification.

8.2 Strengths and limitations

The main strength of the proposed architecture is probably its modularity. Perception, semantic mapping, interaction, planning, and locomotion remain conceptually separated while sharing a common grounded representation of the environment. This makes it possible to replace or improve individual components without redesigning the complete interaction pipeline.

Another strength is the use of a persistent semantic representation rather than direct frame-by-frame interaction. The same internal representation can support different behaviours depending on the user request, including describing perceived entities, reporting their location, reasoning over multiple instances, and navigating towards a semantic target. The interaction layer also follows a conservative strategy in which unsupported information is not directly converted into physical actions.

At the same time, the current prototype presents several limitations. The semantic relations Knowledge Base is manually defined and therefore constrains the set of high-level entities that can be addressed by the interaction parser. Although the persistent perception map contains richer information about the environment, this information is not yet exploited as a fully dynamic source of semantic targets. The dialogue capabilities are also intentionally limited. The LLM interprets each user request independently, but the system does not maintain a complete conversational state for multi-turn referential reasoning. This is particularly evident when the user attempts to clarify an ambiguous navigation target in a subsequent interaction.

8.3 Future improvements

If we started bringing up what's next, we'd ramble on forever, to keep things short, we are going to only touch on the core directions forward.

- **ambiguos navigation goal solver:** A first improvement would be the introduction of a dedicated clarification mechanism for ambiguous references. When multiple objects satisfy a navigation request, the robot could ask the user for additional semantic information, such as a relation, attribute, or nearby object, and use this information to re-ground the target before executing the motion. Before implementing this point we have to check the results given by the improvement of the Semantic Relations Knowledge Base;
- **Semantic Relations KB automatic update:** at that time we demonstrated that we can achieve powerful results with a small, manually defined .yaml file. In the future we could dinamically update this file with the relations learned during dialogues with expert staff and exploration phase by directly exposing the persistent world model to the interaction layer;
- **Hierarchical clusterization:** Instead of representing the environment only as a flat collection of objects, the scene could be organised into higher-level spatial clusters corresponding to rooms, sub-areas, or functional regions. Each perceived object would then be anchored to one of these clusters. For example, a 'chair' associated with *Room 1* would not be considered a valid candidate for an observation occurring in *Room 2*. This hierarchical representation could significantly reduce the search space during data association and simplify object tracking in larger environments, while also improving the **scalability** of the semantic map when multiple rooms or extended office areas are explored;
- **integration of V-JEPA:** V-JEPA was part of the original project idea. The current perception pipeline relies mainly on explicit object detection, segmentation, and semantic association. A future experimental comparison could therefore investigate whether V-JEPA features can support more reliable object association, scene understanding, or temporal reasoning during exploration or not;
- **Sim-To-Real:** Last but not least, the current simulated implementation could be deployed and tested on a physical humanoid platform. Such a transfer would require sensor calibration, hardware-specific control interfaces, safety mechanisms, and further experimental validation of the perception and locomotion modules under real-world conditions. The overall software architecture, however, was deliberately designed so that these hardware-dependent components remain separated from the semantic reasoning and interaction layers;

AI-assisted writing disclosure AI-based language tools were used to support grammar correction and improve the clarity and fluency of the report. All technical content, design decisions, implementations, experiments, and interpretations were produced and validated by the authors.

References

- [1] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024.
- [3] Sara Micol Ferraina, Michele Brienza, Francesco Argenziano, Emanuele Musumeci, Vincenzo Suriani, Domenico D. Bloisi, and Daniele Nardi. Lost-3dsg: Lightweight open-vocabulary 3d scene graphs with semantic tracking in dynamic environments, 2026.
- [4] Matthew Studley, Sarah Carter, Francisco J. Perez-Grau, Antidio Viguria Jiménez, Gabriele Ferri, Fausto Ferreira, Deebul Nair, Sven Schneider, Paul G. Plöger, Pedro U. Lima, Meysam Basiri, Gerhard K. Kraetzschmar, Daniele Nardi, Lun Wang, Emanuele Antonioni, Vincenzo Suriani, and Luca Iocchi. Perspective chapter: European robotics league – benchmarking through smart city robot competitions, 2023.
- [5] Bartneck C., T. Belpaeme, F. Eyssel, T. Kanda, and Keijsers M. and Sabanovic S. Human-robot interaction – an introduction. (2nd edition) cambridge university press, 2024.